Module 2: Recognition Code

CONTENTS PAGE

01	puf_reuse/predict.py	2
02	puf_reuse/model.py	3
03	puf_reuse/requirements.txt	5
04	puf_reuse/examples/example.fasta	6
05	puf_reuse/examples/expected_predictions.csv	7
06	wiki/ablation_experiment/run_ablation.py	8
07	wiki/ablation_experiment/model.py	12





puf_reuse/predict.py

```python
import argparse, joblib
import numpy as np
import pandas as pd
from pathlib import Path
from model import encode, predict

p = argparse.ArgumentParser()
p.add_argument('--model', required=True)
p.add_argument('--fasta', required=True)
p.add_argument('--output', required=True)
a = p.parse_args()
assert not Path(a.output).exists()
records = []
name = None
seq = []
for line in Path(a.fasta).read_text().splitlines():
    line = line.strip()
    if not line:
        continue
    if line.startswith('>'):
        if name is not None:
            records.append((name, ''.join(seq)))
        name = line[1:]
        seq = []
    else:
        if name is None:
            raise ValueError('FASTA header required')
        seq.append(line)
if name is not None:
    records.append((name, ''.join(seq)))
assert (
    records
    and all((r[0] for r in records))
    and (len({r[0] for r in records}) == len(records))
)
b = joblib.load(a.model)
X, score = encode([r[1] for r in records], b['reference'], b['score_logp'])
v = predict(b['fits'], X, score, np.arange(len(records)))
d = pd.DataFrame(
    dict(
        candidate_id=[r[0] for r in records],
        C295_hat=v[:, 0],
        C388_hat=v[:, 1],
        C871_hat=v[:, 2],
        S_hat=v[:, 1] - (v[:, 0] + v[:, 2]) / 2,
        ESM2_score=score,
        classifier_status='untrained: no negative class',
    )
)
d = d.sort_values(['S_hat', 'candidate_id'], ascending=[False, True])
d.insert(0, 'predicted_rank', range(1, len(d) + 1))
d.to_csv(a.output, index=False)
```





puf_reuse/model.py

```
import numpy as np
from scipy.sparse import csr_matrix, hstack
from sklearn.linear_model import Ridge

AA = 'ACDEFGHIKLMNPQRSTVWY'


def encode(sequences, ref, logp):
    rows = []
    cols = []
    vals = []
    scores = []
    for k, s in enumerate(sequences):
        if len(s) != 493 or set(s) - set(AA):
            raise ValueError('Expected 493 canonical amino acids')
        value = 0.0
        for i, (x, y) in enumerate(zip(ref, s)):
            if x != y:
                rows.extend([k, k])
                cols.extend([i * 20 + AA.index(x), i * 20 + AA.index(y)])
                vals.extend([-1.0, 1.0])
                value += float(logp[i, AA.index(y)] - logp[i, AA.index(x)])
        scores.append(value)
    return (
        csr_matrix((vals, (rows, cols)), shape=(len(sequences), 493 * 20)),
        np.array(scores),
    )


def design(onehot, score, indices, name, weight, mu, sd):
    if name == 'OneHot_Ridge':
        return onehot[indices]
    q = csr_matrix(((score[indices] - mu) / sd * weight)[:, None])
    return q if name == 'ESM2_score_Ridge' else hstack([onehot[indices], q], format='csr')

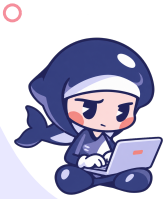

def fit(onehot, score, Y, tr, name, spec):
    fits = []
    for j in range(3):
        ix = np.array(tr)[np.isfinite(Y[np.array(tr), j])]
        mu = float(score[ix].mean())
        sd = max(float(score[ix].std()), 1e-08)
        weight = spec.get('score_weight', 1)
        x = design(onehot, score, ix, name, weight, mu, sd)
        reg = Ridge(alpha=spec['alpha'], solver='lsqr', tol=1e-06)
        reg.fit(x, Y[ix, j])
        fits.append(dict(model=reg, mean=mu, std=sd, name=name, weight=weight))
    return fits


def predict(fits, onehot, score, indices):
    return np.column_stack(
        [
```





```
            f['model'].predict(
                design(
                    onehot, score, indices, f['name'], f['weight'], f['mean'], f['std']
                )
            )
            for f in fits
        ]
    )
```





puf_reuse/requirements.txt

```
numpy==1.26.4
pandas==2.2.3
scipy==1.13.1
scikit-learn==1.5.2
joblib==1.5.3
```





puf_reuse/examples/example.fasta

```
>10_p1_gvd
GRSRLLEDFRNNRYPNLQLREIAGHIMEFSQDQHGGVFIDLKLERATPAERQLVFNEILQAAYQLMVDVFGSYVIRKFFEFGSLEQKLALAERIRGHV
LSLALQMYGCRVIQKALEFIPSDQQNEMVRELDGQVFALSTHPYGSYVIERILEHCLPDQTLPILEELHQHTEQLVQDQYGSYVIEHVLEHGRPEDKS
KIVAEIRGNVLVLSQHKFACRVVQKCVTHASRTERAVLIDELDGQVFALSTHPYGSYVIERILEHCLPDQTLPILEELHQHTEQLVQDQYGSYVIEHV
LEHGRPEDKSKIVAEIRGNVLVLSQHKFANYVVQKCVTHASRTERAVLIDKIRPHTEQLVQDQYGCRVVQHVLEHGRPEDKSKIVAEIRGNVLVLSQH
KFASYVIRKCVTHASRTERAVLIDEVCTMNDGPHSALYTMMKDQYACRVVQKMIDVAEPGQRKIVMHKIRPHIATLRKYTYGKHILAKLEKYYMKNGV
DLG
>11_p1_sne
GRSRLLEDFRNNRYPNLQLREIAGHIMEFSQDQHGSNFIELKLERATPAERQLVFNEILQAAYQLMVDVFGSYVIRKFFEFGSLEQKLALAERIRGHV
LSLALQMYGCRVIQKALEFIPSDQQNEMVRELDGQVFALSTHPYGSYVIERILEHCLPDQTLPILEELHQHTEQLVQDQYGSYVIEHVLEHGRPEDKS
KIVAEIRGNVLVLSQHKFACRVVQKCVTHASRTERAVLIDELDGQVFALSTHPYGSYVIERILEHCLPDQTLPILEELHQHTEQLVQDQYGSYVIEHV
LEHGRPEDKSKIVAEIRGNVLVLSQHKFANYVVQKCVTHASRTERAVLIDKIRPHTEQLVQDQYGCRVVQHVLEHGRPEDKSKIVAEIRGNVLVLSQH
KFASYVIRKCVTHASRTERAVLIDEVCTMNDGPHSALYTMMKDQYACRVVQKMIDVAEPGQRKIVMHKIRPHIATLRKYTYGKHILAKLEKYYMKNGV
DLG
>12_p1_sre
GRSRLLEDFRNNRYPNLQLREIAGHIMEFSQDQHGSRFIELKLERATPAERQLVFNEILQAAYQLMVDVFGSYVIRKFFEFGSLEQKLALAERIRGHV
LSLALQMYGCRVIQKALEFIPSDQQNEMVRELDGQVFALSTHPYGSYVIERILEHCLPDQTLPILEELHQHTEQLVQDQYGSYVIEHVLEHGRPEDKS
KIVAEIRGNVLVLSQHKFACRVVQKCVTHASRTERAVLIDELDGQVFALSTHPYGSYVIERILEHCLPDQTLPILEELHQHTEQLVQDQYGSYVIEHV
LEHGRPEDKSKIVAEIRGNVLVLSQHKFANYVVQKCVTHASRTERAVLIDKIRPHTEQLVQDQYGCRVVQHVLEHGRPEDKSKIVAEIRGNVLVLSQH
KFASYVIRKCVTHASRTERAVLIDEVCTMNDGPHSALYTMMKDQYACRVVQKMIDVAEPGQRKIVMHKIRPHIATLRKYTYGKHILAKLEKYYMKNGV
DLG
```





puf_reuse/examples/expected_predictions.csv

```
predicted_rank,candidate_id,C295_hat,C388_hat,C871_hat,S_hat,ESM2_score,classifier_status
1,10_p1_gvd,0.23296760614152492,0.5378618376456891,0.3798477348976823,0.23145416712608546,-7.39057
.  3441982269,untrained: no negative class
2,11_p1_sne,0.25013567129743375,0.552471802425101,0.4095791502551604,0.22261439164880392,-1.024098
.  3963012695,untrained: no negative class
3,12_p1_sre,0.2559059393295896,0.5612013880092319,0.43140483520556216,0.21754600074165603,4.279422
.  149062157,untrained: no negative class
```





wiki/ablation_experiment/run_ablation.py

```
from pathlib import Path
import argparse, json, time, math, hashlib, platform
import numpy as np, pandas as pd, scipy, sklearn
from scipy.stats import spearmanr, kendalltau
from scipy.sparse import csr_matrix
from model import encode, fit, predict

p = argparse.ArgumentParser()
p.add_argument('--limit', type=int, default=66)
p.add_argument(
    '--output', type=Path, help='New results directory; omit to resume bundled results'
)
a = p.parse_args()
root = Path(__file__).resolve().parent
inp = root / 'inputs'
out = a.output.resolve() if a.output else root / 'results'
out.mkdir(parents=True, exist_ok=True)
cfg = json.loads((root / 'protocol.json').read_text())
for name, expected in cfg['inputs'].items():
    assert (
        hashlib.sha256((inp / name).read_bytes()).hexdigest() == expected
    ), f'Input changed: {name}; this protocol reproduces the fixed dataset only'
mf = pd.read_csv(inp / 'construct_manifest.csv')
ta = pd.read_csv(inp / 'construct_targets.csv')
assert mf.construct_id.tolist() == ta.construct_id.tolist()
Y = ta[['C295', 'C388', 'C871']].to_numpy()
ref = json.loads((inp / 'reference.json').read_text())['reference']
logp = np.load(inp / 'reference_masked_logp.npy')
onehot, score = encode(mf.puf_sequence.tolist(), ref, logp)
pos = csr_matrix(
    np.array([[a != b for a, b in zip(s, ref)] for s in mf.puf_sequence], float)
)
groups = mf.leakage_group.to_numpy()
folds = json.loads((inp / 'splits.json').read_text())
assert len(mf) == 81 and np.isfinite(Y).all(1).sum() == 66 and (len(ref) == 493)
assert (pos.sum(1).A.ravel() == onehot.getnnz(1) / 2).all()


def S(y):
    return y[:, 1] - (y[:, 0] + y[:, 2]) / 2


def obj(y, q, ids):
    t = S(y)
    e = S(q)
    rho = float(spearmanr(t, e).statistic)
    rho = rho if np.isfinite(rho) else -1.0
    k = math.ceil(15 / 66 * len(t))
    sel = np.lexsort((ids, -e))[:k]
    return [rho, float(t[sel].mean()), -float(np.mean((y - q) ** 2))]


start = time.time()
rows = []
mnames = [
    'Position_only_Ridge',
```





```
    'OneHot_Ridge',
    'ESM2_score_Ridge',
    'Hsu_ESM2_OneHot_Ridge',
]
for k, f in enumerate(folds[: a.limit]):
    tr = np.array(f['train'])
    te = np.array(f['test'])
    assert not set(groups[tr]) & set(groups[te])
    assert set(tr) == set(np.flatnonzero(~np.isin(groups, groups[te])))
    path = out / f'fold_{k + 1:03d}.json'
    if path.exists():
        rr = json.loads(path.read_text())
    else:
        rr = {}
        for name in mnames:
            X = pos if name == 'Position_only_Ridge' else onehot
            key = 'OneHot_Ridge' if name == 'Position_only_Ridge' else name
            candidates = []
            for alpha in cfg['alphas']:
                for weight in (
                    cfg['score_weights'] if name == 'Hsu_ESM2_OneHot_Ridge' else [1]
                ):
                    spec = dict(alpha=alpha, score_weight=weight)
                    vals = []
                    for sp in f['inner']:
                        ti = np.array(sp['train81'])
                        vi = np.array(sp['validation66'])
                        assert not set(groups[ti]) & set(groups[vi])
                        assert set(ti) <= set(tr) and set(vi) <= set(tr)
                        q = predict(fit(X, score, Y, ti, key, spec), X, score, vi)
                        vals.append(obj(Y[vi], q, vi))
                    candidates.append(
                        dict(spec=spec, metrics=np.mean(vals, axis=0).tolist())
                    )
            best = max(candidates, key=lambda x: tuple(x['metrics']))
            q = predict(fit(X, score, Y, tr, key, best['spec']), X, score, te)
            assert np.isfinite(q).all()
            rr[name] = dict(selected=best, candidates=candidates, predictions=q.tolist())
        path.write_text(json.dumps(rr, indent=2))
    for name in mnames:
        for i, q in zip(te, rr[name]['predictions']):
            r = dict(
                construct_id=mf.iloc[i].construct_id,
                batch=mf.iloc[i].batch,
                double_repeat=len(json.loads(mf.iloc[i].repeat_position_set)) > 1,
                model=name + '_A',
                fold=k + 1,
                S=float(S(Y[i : i + 1])[0]),
                S_hat=float(S(np.array([q]))[0]),
            )
            for j, c in enumerate(['C295', 'C388', 'C871']):
                r[c] = Y[i, j]
                r[c + '_hat'] = q[j]
            rows.append(r)
    print('fold', k + 1, 'seconds', round(time.time() - start, 2), flush=True)
if a.limit < 66:
    print('PILOT PASS')
    raise SystemExit
for i in np.flatnonzero(np.isfinite(Y).all(1)):
    rows.append(
        dict(
```





```
            construct_id=mf.iloc[i].construct_id,
            batch=mf.iloc[i].batch,
            double_repeat=len(json.loads(mf.iloc[i].repeat_position_set)) > 1,
            model='ESM2_score_zero_shot',
            fold='',
            S=float(S(Y[i : i + 1])[0]),
            S_hat=score[i],
        )
    )
df = pd.DataFrame(rows)
metrics = []
ranked = []
for name, d in df.groupby('model'):
    assert len(d) == 66
    d = d.sort_values(['S_hat', 'construct_id'], ascending=[False, True])
    d['predicted_rank66'] = range(1, 67)
    d['measured_rank66'] = d.S.rank(ascending=False, method='min').astype(int)
    ranked.append(d)
    for subset in ['all', 'batch1', 'batch2', 'double_repeat']:
        e = (
            d
            if subset == 'all'
            else (
                d[d.double_repeat] if subset == 'double_repeat' else d[d.batch.eq(subset)]
            )
        )
        n = len(e)
        m = dict(
            model=name,
            subset=subset,
            n=n,
            Spearman=spearmanr(e.S, e.S_hat).statistic,
            Kendall=kendalltau(e.S, e.S_hat).statistic,
        )
        if n >= 30:
            pred = e.head(30)
            truth = e.sort_values(['S', 'construct_id'], ascending=[False, True]).head(30)
            h = len(set(pred.construct_id) & set(truth.construct_id))
            m.update(
                Top30_overlap=h,
                Top30_hit_rate=h / 30,
                Top30_random_expected=900 / n,
                Top30_enrichment=h / (900 / n),
                Top30_mean_S=pred.S.mean(),
            )
        metrics.append(m)
ranked = pd.concat(ranked)
ranked.to_csv(out / 'oof_ablation.csv', index=False)
pd.DataFrame(metrics).to_csv(out / 'ablation_metrics.csv', index=False)
old = pd.read_csv(inp / 'oof_all_models.csv')
diff = {}
for name in [
    'OneHot_Ridge_A',
    'ESM2_score_Ridge_A',
    'Hsu_ESM2_OneHot_Ridge_A',
    'ESM2_score_zero_shot',
]:
    d = ranked[ranked.model.eq(name)].merge(
        old[old.model.eq(name)], on='construct_id', suffixes=('_new', '_old')
    )
    delta = float(abs(d.S_hat_new - d.S_hat_old).max())
```





```
    assert delta < 1e-07, (name, delta)
    diff[name] = delta
(out / 'validation.json').write_text(
    json.dumps(
        dict(
            seconds=time.time() - start,
            folds=66,
            OOF_rows=len(ranked),
            group_leakage_checks=True,
            old_predictions_max_difference=diff,
            software=dict(
                python=platform.python_version(),
                numpy=np.__version__,
                scipy=scipy.__version__,
                sklearn=sklearn.__version__,
            ),
        ),
        indent=2,
    )
)
print(pd.DataFrame(metrics).query("subset=='all'").to_string(index=False))
```





wiki/ablation_experiment/model.py

```python
import numpy as np
from scipy.sparse import csr_matrix, hstack
from sklearn.linear_model import Ridge

AA = 'ACDEFGHIKLMNPQRSTVWY'


def encode(sequences, ref, logp):
    rows = []
    cols = []
    vals = []
    scores = []
    for k, s in enumerate(sequences):
        if len(s) != 493 or set(s) - set(AA):
            raise ValueError('Expected 493 canonical amino acids')
        value = 0.0
        for i, (x, y) in enumerate(zip(ref, s)):
            if x != y:
                rows.extend([k, k])
                cols.extend([i * 20 + AA.index(x), i * 20 + AA.index(y)])
                vals.extend([-1.0, 1.0])
                value += float(logp[i, AA.index(y)] - logp[i, AA.index(x)])
        scores.append(value)
    return (
        csr_matrix((vals, (rows, cols)), shape=(len(sequences), 493 * 20)),
        np.array(scores),
    )


def design(onehot, score, indices, name, weight, mu, sd):
    if name == 'OneHot_Ridge':
        return onehot[indices]
    q = csr_matrix(((score[indices] - mu) / sd * weight)[:, None])
    return q if name == 'ESM2_score_Ridge' else hstack([onehot[indices], q], format='csr')


def fit(onehot, score, Y, tr, name, spec):
    fits = []
    for j in range(3):
        ix = np.array(tr)[np.isfinite(Y[np.array(tr), j])]
        mu = float(score[ix].mean())
        sd = max(float(score[ix].std()), 1e-08)
        weight = spec.get('score_weight', 1)
        x = design(onehot, score, ix, name, weight, mu, sd)
        reg = Ridge(alpha=spec['alpha'], solver='lsqr', tol=1e-06)
        reg.fit(x, Y[ix, j])
        fits.append(dict(model=reg, mean=mu, std=sd, name=name, weight=weight))
    return fits


def predict(fits, onehot, score, indices):
    return np.column_stack(
        [
```





```
            f['model'].predict(
                design(
                    onehot, score, indices, f['name'], f['weight'], f['mean'], f['std']
                )
            )
            for f in fits
        ]
    )
```

